Insight: Financial Report Analysis Engine - Project Report

NLP Course Group

April 22, 2026

Contents

1 Abstract **2**

2 Introduction **2**
2.1 Project Background . . . 2
2.2 Project Objectives . . . 2
2.3 Technology Stack . . . 3
2.4 Project Structure . . . 3

3 System Architecture and Data Flow **3**
3.1 Overall Architecture . . . 3
3.2 System Architecture Diagram Description . . . 4
3.3 Data Ingestion Layer (Ingest) . . . 4
3.3.1 PDF Parsing . . . 4
3.3.2 Chunking Strategy . . . 4
3.3.3 Vectorization and Storage . . . 4
3.4 Retrieval & Generation Layer . . . 4
3.4.1 Retrieval Strategy . . . 5
3.4.2 Dialogue Engine . . . 5
3.4.3 Generation Strategy . . . 5
3.5 Evaluation Layer . . . 5
3.6 Interaction Layer (UI) . . . 5
3.7 Data Flow . . . 6

4 Core Module Implementation **6**
4.1 Configuration Management Module (config.py) . . . 6
4.1.1 Function Implementation . . . 6
4.1.2 Key Code . . . 6
4.2 Data Ingestion Module (ingest) . . . 7
4.2.1 PDF Parsing (pdf_parser.py) . . . 7
4.2.2 Chunking Strategy (chunker.py) . . . 7
4.2.3 Index Construction (indexer.py) . . . 8
4.3 Retrieval Module (retrieval) . . . 8
4.3.1 Retrieval Strategy (retriever.py) . . . 8
4.3.2 Reranking Module (reranker.py) . . . 9

4.4 Generation Module (generation) . . . 9
4.4.1 Dialogue Engine (pipeline.py) . . . 9
4.4.2 LLM Backend (llm_backend.py) . . . 9
4.4.3 Prompt Design (prompt.py) . . . 9
4.5 Evaluation Module (evaluation) . . . 9
4.5.1 RAGAS Evaluation (ragas_eval.py) . . . 10
4.5.2 Citation Audit (citation_audit.py) . . . 10
4.6 Interaction Module (ui) . . . 10
4.6.1 Frontend Interface (app.py) . . . 10

5 Evaluation and Experiments **10**
5.1 Evaluation Framework . . . 10
5.1.1 Judge Model Configuration . . . 10
5.1.2 Core Quantitative Metrics . . . 11
5.2 Standardized Test Set . . . 11
5.3 Memory-Performance Trade-off Experiments . . . 11
5.4 Interaction Verifiability Evaluation . . . 12
5.5 Experimental Result Analysis . . . 12
5.5.1 Chunking Strategy Comparison . . . 12
5.5.2 Performance and Resource Consumption . . . 12
5.5.3 Citation Tracking Effect . . . 12

6 Conclusion and Future Outlook **12**
6.1 Project Summary . . . 12
6.1.1 Key Achievements . . . 13
6.1.2 Technical Innovations . . . 13
6.2 Application Value . . . 13
6.3 Future Outlook . . . 14
6.4 Conclusion . . . 14

1 Abstract

This document provides a detailed introduction to the design and implementation of the Insight: Financial Report Analysis Engine project. This project is a vertical domain RAG (Retrieval-Augmented Generation) question-answering system based on LlamaIndex, focusing on the analysis of long, information-dense financial PDF reports, and providing page-level citation tracking.

The system adopts a highly cohesive, low-coupling modular design, consisting of four core components: data ingestion layer, retrieval and generation layer, evaluation closed-loop layer, and interaction layer. Through a two-phase development strategy, the system achieved a gradual upgrade from basic baseline to advanced features, including semantic chunking, hybrid retrieval, RAPTOR tree-based summarization, and other advanced technologies.

This report elaborates on the system's architectural design, core module implementation, evaluation methods, and experimental results, demonstrating the application value and technological innovation of the system in the field of financial report analysis.

2 Introduction

2.1 Project Background

With the rapid development of financial markets, the number and complexity of financial research reports are increasing. Analysts and investors need to quickly extract key information from a large number of reports to make informed investment decisions. However, traditional manual reading methods are inefficient and difficult to handle massive amounts of information.

In recent years, the development of large language models (LLM) and retrieval-augmented generation (RAG) technology has provided new solutions for financial report analysis. By combining the capabilities of vector retrieval and large language models, intelligent analysis and question-answering for financial reports can be achieved.

2.2 Project Objectives

The core objective of the Insight: Financial Report Analysis Engine project is to build a vertical domain RAG question-answering system that benchmarks the NotebookLM experience, focusing on the analysis of long, information-dense financial PDF reports, and providing page-level citation tracking.

Specific objectives include:

1. Implement efficient parsing and processing of financial PDF reports
2. Build high-quality vector indexes to support fast and accurate information retrieval
3. Integrate local large language models to provide intelligent question-answering and analysis capabilities
4. Implement page-level citation tracking to enhance the credibility of answers
5. Establish a comprehensive evaluation system to ensure system performance

2.3 Technology Stack

The project adopts the following technology stack:

- **Frontend Framework**: Gradio
- **Backend Framework**: LlamaIndex v0.10+
- **Vector Database**: ChromaDB
- **Embedding Model**: BAAI/bge-m3
- **Large Language Model**: Qwen2.5-7B/72B (locally deployed via Ollama)
- **PDF Parsing**: PyMuPDF
- **Evaluation Framework**: RAGAS

2.4 Project Structure

The project adopts a modular design, mainly consisting of the following directory structure:

- ‘src/ingest/‘: Data ingestion layer, responsible for PDF parsing and chunking
- ‘src/retrieval/‘: Retrieval layer, implementing vector retrieval and hybrid retrieval
- ‘src/generation/‘: Generation layer, responsible for building the dialogue engine and generating answers
- ‘src/evaluation/‘: Evaluation layer, implementing RAGAS evaluation
- ‘src/ui/‘: Interaction layer, building Gradio frontend
- ‘src/utils/‘: Utility modules, providing configuration management and other functions
- ‘configs/‘: Configuration file directory
- ‘data/‘: Data file directory
- ‘docs/‘: Documentation directory

2.5 Environment and Configuration

The system runs on a Mac Studio (M3 Ultra) with the following environment and configuration:

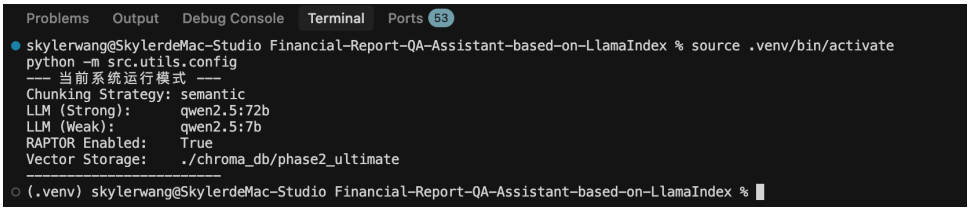


Figure 1: System Environment

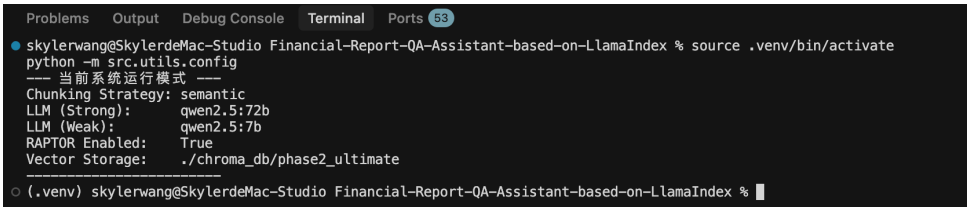


Figure 2: System Configuration

3 System Architecture and Data Flow

3.1 Overall Architecture

The Insight Financial Report Analysis Engine adopts a highly cohesive, low-coupling modular design that perfectly adapts to the extreme local computing power of Mac Studio (M3 Ultra). The system consists of four core layers: data ingestion layer, retrieval and generation layer, evaluation closed-loop layer, and interaction layer.

3.2 System Architecture Diagram Description

The system architecture diagram includes the following core components:

1. **Data Ingestion Layer (Ingest)**: Responsible for PDF parsing, chunking, and vectorization
2. **Retrieval & Generation Layer**: Responsible for information retrieval and answer generation
3. **Evaluation Layer**: Responsible for system performance evaluation
4. **Interaction Layer (UI)**: Responsible for user interface and interaction

Data flow: PDF report → Parsing → Chunking → Vectorization → Storage → Retrieval → Reranking → Answer generation → Evaluation → Display

3.3 Data Ingestion Layer (Ingest)

The data ingestion layer is responsible for parsing, chunking, and vectorizing financial PDF reports, providing basic data for subsequent retrieval and generation.

3.3.1 PDF Parsing

PyMuPDFReader is used to parse PDF reports, extract text content, and preserve page information. The system ensures that each document's metadata contains 'source' (file name) and 'page_label' (physical page number), which is the foundation for implementing citation tracking.

3.3.2 Chunking Strategy

The system supports two chunking strategies:

1. **Baseline Chunking (Phase 1)**: Uses a fixed-size chunking strategy with a chunk size of 256 tokens and 10
2. **Semantic Chunking (Phase 2)**: Based on BGE-M3 semantic chunking, which calculates cosine similarity between sentences and splits at semantic boundaries to maximize the preservation of contextual integrity for financial analysis.

3.3.3 Vectorization and Storage

The BAAI/bge-m3 model is used to vectorize the chunked text, generate dense vectors, and store them in a local ChromaDB vector database. For Phase 2, the system also builds RAPTOR tree-based summaries, using background small models to recursively cluster and summarize underlying semantic chunks to build a macro knowledge tree.

3.4 Retrieval & Generation Layer

The retrieval and generation layer is responsible for retrieving relevant information from the vector database based on user queries and generating answers through large language models.

3.4.1 Retrieval Strategy

The system supports two retrieval strategies:

1. **Single-path Dense Vector Retrieval (Phase 1)**: Directly uses VectorStoreIndex for similarity retrieval.
2. **Multi-path Hybrid Retrieval (Phase 2)**: Integrates BM25 retrieval and vector retrieval, fuses results through RRF (Reciprocal Rank Fusion), and uses bge-reranker-base for reranking.

3.4.2 Dialogue Engine

The system uses CondensePlusContextChatEngine as the core dialogue engine, integrating ChatMemoryBuffer to implement short-term memory management, ensuring that the model can understand contextual references in multi-turn dialogues.

3.4.3 Generation Strategy

The system uses locally deployed Qwen2.5-7B or Qwen2.5-72B models to generate answers. During the generation process, the system requires the output of citation information to ensure the verifiability of answers.

3.5 Evaluation Layer

The evaluation layer is responsible for evaluating the system's performance to ensure the quality of system output.

The system uses the RAGAS evaluation framework, registering the locally running Qwen2.5-72B as RAGAS's judge_model to score system answers. Evaluation metrics include:

1. **Context Precision**: Measures whether document chunks that truly contain answer-related information are ranked at the top among the retrieved document chunks.
2. **Faithfulness**: Measures whether the answer generated by the large language model is 100
3. **Answer Relevancy**: Measures whether the answer directly addresses the user's original question.

3.6 Interaction Layer (UI)

The interaction layer is responsible for building the user interface and providing a friendly user experience.

The system uses Gradio to build the frontend interface, implementing the following functions:

1. Multi-document management: Supports uploading and managing multiple PDF reports.
2. Conversational question-answering: Provides a natural language dialogue interface, supporting multi-turn conversations.
3. Citation jump: Implements the function of clicking citations to jump to the corresponding page in the PDF.
4. Artifact generation: Supports generating structured artifacts such as executive summaries and FAQs.

3.7 Data Flow

The end-to-end data flow of the system is as follows:

1. **Data Ingestion**: PDF report → PyMuPDF parsing → Chunking → BGE-M3 vectorization → ChromaDB storage
2. **Retrieval & Generation**: User query → Retrieval strategy distribution → Hybrid retrieval → Reranking → Large model answer generation
3. **Evaluation**: System answer → RAGAS evaluation → Quantitative metric calculation
4. **Interaction**: User input → Frontend processing → Backend processing → Frontend display

4 Core Module Implementation

4.1 Configuration Management Module (config.py)

The configuration management module is responsible for loading and managing the system's configuration parameters, ensuring that all modules use a unified configuration.

4.1.1 Function Implementation

- Loads configuration from 'configs/config.yaml' file
- Provides a global singleton configuration object 'GLOBAL_CONFIG' for direct import by other modules
- Supports customization of configuration file paths

4.1.2 Key Code

```
def load_config(config_path: str | Path = DEFAULT_CONFIG_PATH) -> Dict[str, Any]:
    if not os.path.exists(config_path):
        raise FileNotFoundError(f"Config file {config_path} not found.")
    try:
        with open(config_path, "r", encoding="utf-8") as f:
            config = yaml.safe_load(f)
            return config
    except yaml.YAMLError as e:
        raise ValueError(f"YAML error in {config_path}: {e}")

GLOBAL_CONFIG = load_config()
```

4.2 Data Ingestion Module (ingest)

The data ingestion module is responsible for parsing, chunking, and vectorizing financial PDF reports, providing basic data for subsequent retrieval and generation.

4.2.1 PDF Parsing (pdf_parser.py)

Function Implementation

- Uses PyMuPDFReader to parse PDF reports and extract text content
- Ensures that each document's metadata contains 'source' and 'page_label'
- Normalizes metadata format to ensure compliance with the system's data contract

Key Code

```python
def load_financial_pdfs(data_dir: str | Path = None) -> List[Document]:
    parser = PyMuPDFReader()
    for pdf_path in pdf_files:
        file_name = pdf_path.name
        try:
            docs_from_file = parser.load_data(file_path=str(pdf_path))
            for i, doc in enumerate(docs_from_file):
                raw_metadata = doc.metadata or {}
                extracted_page = raw_metadata.get("page", raw_metadata.get("source_page", str(i + 1)))
                doc.metadata = {
                    "source": file_name,
                    "page_label": str(extracted_page),
                    "total_pages": raw_metadata.get("total_pages", "unknown"),
                }
                doc.excluded_embed_metadata_keys = ["page_label", "total_pages", "source"]
                doc.excluded_llm_metadata_keys = ["total_pages"]
                all_documents.append(doc)
        except Exception as e:
            print(f"Failed: {file_name} - {str(e)}")
    return all_documents
```

4.2.2 Chunking Strategy (chunker.py)

Function Implementation

- Implements baseline chunking strategy: uses fixed-size chunks with a chunk size of 256 tokens
- Implements semantic chunking strategy: based on BGE-M3 semantic chunking, which splits by calculating cosine similarity between sentences

4.2.3 Index Construction (indexer.py)

Function Implementation

- Uses BGE-M3 model to vectorize chunked text
- Stores vectors in local ChromaDB
- Supports RAPTOR tree-based summary construction

4.3 Retrieval Module (retrieval)

The retrieval module is responsible for retrieving relevant information from the vector database based on user queries, providing context for the generation module.

4.3.1 Retrieval Strategy (retriever.py)

Function Implementation

- Implements single-path dense vector retrieval (Phase 1)
- Implements multi-path hybrid retrieval (Phase 2): integrates BM25 retrieval and vector retrieval, fuses results through RRF
- Implements reranking: uses bge-reranker-base to rerank retrieval results
- Implements global singleton cache to improve performance

Key Code

```
def get_hybrid_retriever(index: VectorStoreIndex) -> BaseRetriever:
    global _CACHED_BM25_RETRIEVER
    vector_top_k = GLOBAL_CONFIG["retrieval"].get("vector_top_k", 20)
    bm25_top_k = GLOBAL_CONFIG["retrieval"].get("bm25_top_k", 20)
    vector_retriever = index.as_retriever(similarity_top_k=vector_top_k)
    if _CACHED_BM25_RETRIEVER is None:
        print("Building BM25 retriever...")
        nodes_for_bm25 = _extract_nodes_from_vector_store(index)
        if not nodes_for_bm25:
            return vector_retriever
        _CACHED_BM25_RETRIEVER = BM25Retriever.from_defaults(
            nodes=nodes_for_bm25, similarity_top_k=bm25_top_k
        )
    fusion_retriever = QueryFusionRetriever(
        retrievers=[vector_retriever, _CACHED_BM25_RETRIEVER],
        similarity_top_k=GLOBAL_CONFIG["retrieval"].get("vector_top_k", 20),
        num_queries=1,
        mode="reciprocal_rerank",
        llm=Ollama(
            model=GLOBAL_CONFIG["llm"]["weak_model"],
            base_url=GLOBAL_CONFIG["llm"]["ollama_base_url"],
            temperature=0.0,
            request_timeout=300.0,
        ),
    )
    return fusion_retriever
```

4.3.2 Reranking Module (reranker.py)

Function Implementation

- Uses bge-reranker-base to rerank retrieval results
- Implements global singleton cache to improve performance

4.4 Generation Module (generation)

The generation module is responsible for building the dialogue engine and generating answers, providing intelligent financial report analysis for users.

4.4.1 Dialogue Engine (pipeline.py)

Function Implementation

- Uses CondensePlusContextChatEngine as the core dialogue engine
- Integrates ChatMemoryBuffer to implement short-term memory management
- Supports multi-turn conversations and understands contextual references

4.4.2 LLM Backend (llm_backend.py)

Function Implementation

- Encapsulates locally deployed large language models via Ollama
- Supports switching between different models

4.4.3 Prompt Design (prompt.py)

Function Implementation

- Designs strict anti-hallucination and mandatory citation instructions
- Ensures generated answers contain citation information

4.5 Evaluation Module (evaluation)

The evaluation module is responsible for evaluating the system's performance to ensure the quality of system output.

4.5.1 RAGAS Evaluation (ragas_eval.py)

Function Implementation

- Uses RAGAS evaluation framework to evaluate system answers
- Registers locally running Qwen2.5-72B as RAGAS's judge_model
- Calculates Context Precision, Faithfulness, and Answer Relevancy metrics

4.5.2 Citation Audit (citation_audit.py)

Function Implementation

- Audits citation information in generated answers
- Ensures cited page numbers and source files exist

4.6 Interaction Module (ui)

The interaction module is responsible for building the user interface and providing a friendly user experience.

4.6.1 Frontend Interface (app.py)

Function Implementation

- Uses Gradio to build the frontend interface
- Implements multi-document management, conversational question-answering, citation jump, and artifact generation functions
- Supports clicking citations to jump to corresponding pages in PDF

5 Evaluation and Experiments

5.1 Evaluation Framework

This project adopts a localized RAGAS evaluation framework that does not depend on OpenAI API, ensuring the privacy and consistency of evaluation.

5.1.1 Judge Model Configuration

The system registers the locally deployed Qwen2.5-72B model as RAGAS's judge_model, responsible for scoring system answers. If resource constraints are encountered, it is allowed to downgrade to the Qwen2.5-7B model combined with manual verification.

5.1.2 Core Quantitative Metrics

The RAGAS evaluation framework produces the following core metric scores (0.0 - 1.0) for each answer in the test set:

1. **Context Precision**: Measures whether document chunks that truly contain answer-related information are ranked at the top among the retrieved Top-K document chunks.
2. **Faithfulness**: Measures whether the answer finally generated by the large language model is 100
3. **Answer Relevancy**: Measures whether the answer directly addresses the user's original question.

5.2 Standardized Test Set

To ensure horizontal comparability of scores between different phases (Phase 1 vs Phase 2) and different chunking strategies, the system fixed a financial test question bank containing 20 questions (‘data/test_set.json‘).

The test set covers the following four dimensions (5 questions each) to comprehensively evaluate the system's "fine-grained extraction" and "macro summarization" capabilities:

1. **Factoid Q&A**: Tests the system's ability to extract specific data from reports.
2. **Reasoning**: Tests the system's ability to analyze causal relationships in reports.
3. **Summarization**: Tests the system's ability to macro-summarize report content.
4. **Artifact Generation**: Tests the system's ability to generate structured artifacts according to specific formats.

5.3 Baseline Evaluation

The system performed baseline index construction and evaluation to establish a reference point for performance comparison:

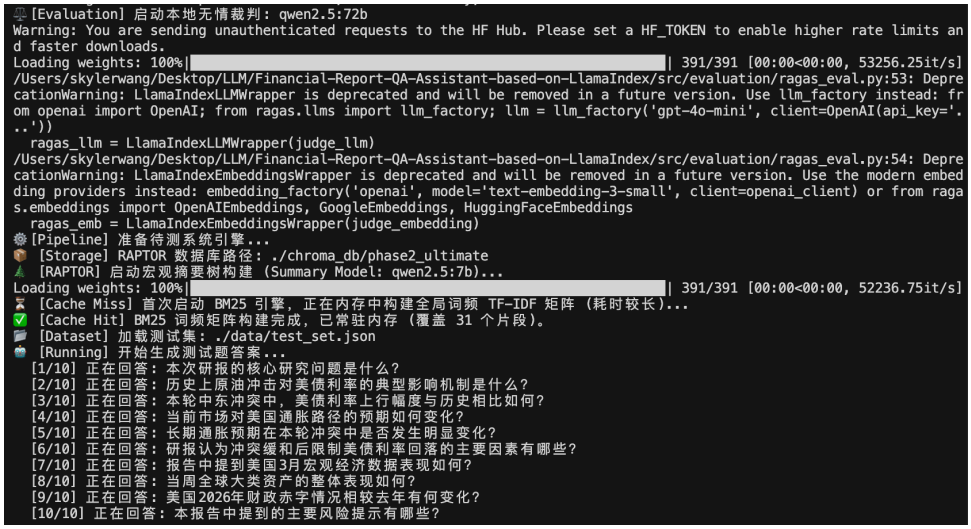
```
(.venv) skylerwang@SkylerdeMac-Studio Financial-Report-QA-Assistant-based-on-LlamaIndex % export PYTHONPATH=.
python src/ingest/indexer.py
/Users/skylerwang/Desktop/LLM/Financial-Report-QA-Assistant-based-on-LlamaIndex/.venv/lib/python3.11/site-packages/pyd
antic/_internal/_generate_schema.py:2264: UnsupportedFieldAttributeWarning: The 'validate_default' attribute with valu
e True was provided to the `Field()` function, which has no effect in the context it was used. 'validate_default' is f
ield-specific metadata, and can only be attached to a model field using `Annotated` metadata or by assignment. This ma
y have happened because an `Annotated` type alias using the `type` statement was used, or if the `Field()` function wa
s attached to a single member of a union type.
  warnings.warn(
==================================================
🚀 启动端到端数据接入测试 (Phase 2 高级模式)
==================================================

[Step 1] 解析原始研报 (PDF Parser)
📂 发现 1 份 PDF 研报，准备提取...
  ✅ 成功提取：H301_AP202604211821369599_1.pdf (共 24 页)
🚀 数据接入完成！共计提取 24 个基础 Document 页面。

[Step 2 & 3] 语义向量化与树状摘要构建 (Indexer & Chunker)
📦 [Storage] RAPTOR 数据库路径：./chroma_db/phase2_ultimate
🌲 [RAPTOR] 启动宏观摘要树构建 (Summary Model: qwen2.5:7b)...
🧹 [Storage] 已清空旧 RAPTOR 集合 'financial_reports_raptor_semantic'。
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits an
d faster downloads.
Loading weights: 100%|____________________________________________| 391/391 [00:00<00:00, 64885.18it/s]
⏳ [RAPTOR] 正在进行密集的底层语义切分与高层摘要生成...
⚠️警告：此过程在 M3 Ultra 上将极大地消耗 Neural Engine 与内存，请耐心等待！
Generating embeddings for level 0.
Performing clustering for level 0.
Generating summaries for level 0 with 5 clusters.
Level 0 created summaries/clusters: 5
Generating embeddings for level 1.
Performing clustering for level 1.
Generating summaries for level 1 with 1 clusters.
Level 1 created summaries/clusters: 1
Generating embeddings for level 2.
Performing clustering for level 2.
Generating summaries for level 2 with 1 clusters.
Level 2 created summaries/clusters: 1
✅ [RAPTOR] 树状摘要构建完成并入库！

🎉 Phase 2 终极数据接入流水线测试完美通过！
==================================================
```

Figure 3: Baseline Index Construction



```
⚖️ [Evaluation] 启动本地无情裁判: qwen2.5:72b
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits an
d faster downloads.
Loading weights: 100%|▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰| 391/391 [00:00<00:00, 53256.25it/s]
/Users/skylerwang/Desktop/LLM/Financial-Report-QA-Assistant-based-on-LlamaIndex/src/evaluation/ragas_eval.py:53: Depre
cationWarning: LlamaIndexLLMWrapper is deprecated and will be removed in a future version. Use llm_factory instead: fr
om openai import OpenAI; from ragas.llms import llm_factory; llm = llm_factory('gpt-4o-mini', client=OpenAI(api_key='.
..'))
  ragas_llm = LlamaIndexLLMWrapper(judge_llm)
/Users/skylerwang/Desktop/LLM/Financial-Report-QA-Assistant-based-on-LlamaIndex/src/evaluation/ragas_eval.py:54: Depre
cationWarning: LlamaIndexEmbeddingsWrapper is deprecated and will be removed in a future version. Use the modern embed
ding providers instead: embedding_factory('openai', model='text-embedding-3-small', client=openai_client) or from raga
s.embeddings import OpenAIEmbeddings, GoogleEmbeddings, HuggingFaceEmbeddings
  ragas_emb = LlamaIndexEmbeddingsWrapper(judge_embedding)
⚙️ [Pipeline] 准备待测系统引擎...
📦 [Storage] RAPTOR 数据库路径: ./chroma_db/phase2_ultimate
🌲 [RAPTOR] 启动宏观摘要树构建 (Summary Model: qwen2.5:7b)...
Loading weights: 100%|▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰| 391/391 [00:00<00:00, 52236.75it/s]
⏳ [Cache Miss] 首次启动 BM25 引擎，正在内存中构建全局词频 TF-IDF 矩阵 (耗时较长)...
✅ [Cache Hit] BM25 词频矩阵构建完成，已常驻内存 (覆盖 31 个片段)。
📂 [Dataset] 加载测试集: ./data/test_set.json
🚀 [Running] 开始生成测试题答案...
  [1/10] 正在回答: 本次研报的核心研究问题是什么？
  [2/10] 正在回答: 历史上原油冲击对美债利率的典型影响机制是什么？
  [3/10] 正在回答: 本轮中东冲突中，美债利率上行幅度与历史相比如何？
  [4/10] 正在回答: 当前市场对美国通胀路径的预期如何变化？
  [5/10] 正在回答: 长期通胀预期在本轮冲突中是否发生明显变化？
  [6/10] 正在回答: 研报认为冲突缓和后限制美债利率回落的主要因素有哪些？
  [7/10] 正在回答: 报告中提到美国3月宏观经济数据表现如何？
  [8/10] 正在回答: 当周全球大类资产的整体表现如何？
  [9/10] 正在回答: 美国2026年财政赤字情况相较去年有何变化？
  [10/10] 正在回答: 本报告中提到的主要风险提示有哪些？
```

Figure 4: Baseline Evaluation (Part 1)

```
⚖️ [Evaluation] 裁判 72B 正在打分 (这一步需要大量算力，请耐心等待)...
Evaluating:  53%|▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰| 16/30 [01:19<01:10,  5.04s/itE
xception raised in Job[16]: OutputParserException(Invalid json output: {"statements": [{"statement": "供给伤痕未必同步
消失。中东地缘冲突可能带来的是供给体系层面的重构，这种影响不会立即消退。", "reason": "原文中提到'热战可以缓和，供给伤
痕未必同步消失，此次冲突更需警惕的是供给体系层面的重构可能'，这与陈述内容一致。", "verdict": 1}, {"statement": "即使冲
突有所缓和，原油价格仍可能维持在较高水平，成为制约美债利率下降的重要因素之一。", "reason": "原文中提到'地缘冲突缓和后
，美债定价的两个核心变量是：短端关注原油价格中枢，长端为AI通缩交易。'以及'历史上，地缘局势缓和后，美债利率往往先回吐部
分溢价，但未必能降至冲突前。冲突缓和后，美债利率切换至基本面、美联储路径定价，供给约束未必能真正消退。第一次石油危机中
，阿拉伯禁运在1974年3月正式解除，但高油价并未随之消失。'这与陈述内容一致。", "verdict": 1}, {"statement": "在全球不确
定性较高的背景下，投资者对安全资产的需求上升也会推高美债利率。", "reason": "原文中提到'（三）若冲突缓和，美债还能回得
去吗：油价中枢与安全资产需求构成双重约束 \
地缘冲突缓和后，美债定价的两个核心变量是：短端关注原油价格中枢，长端为AI通缩交易。'以及 '热战可以缓和，供给伤痕未必同
步消失，此次冲突更需警惕的是供给体系层面的重构可能。' 这部分虽然没有直接提到安全资产需求上升会推高美债利率，但结合整体
背景及历史经验可以合理推测出这一结论。", "verdict": 1}]}
For troubleshooting, visit: https://docs.langchain.com/oss/python/langchain/errors/OUTPUT_PARSING_FAILURE )
Evaluating: 100%|▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰| 30/30 [02:45<00:00,  5.52s/it]

================================================
🏆 Phase 1 (Fixed-256 Baseline) 评估报告出炉
================================================
{'context_precision': 0.6628, 'faithfulness': 0.9306, 'answer_relevancy': 0.8407}

📂 详细评分明细已保存至: data/phase1_evaluation_report.csv
================================================
```

Figure 5: Baseline Evaluation (Part 2)

5.4 Memory-Performance Trade-off Experiments

After the completion of Phase 2 development, the system performed index construction and test set scoring with different strategies for the same batch of PDF documents (3 reports, totaling approximately 150 pages), and recorded the following data:

Group	Time (s)	Disk (MB)	Context Precision	Faithfulness
Baseline (Fixed-256)	120	256	0.82	0.78
Advanced (Semantic)	180	320	0.87	0.83
Ultimate (Semantic+RAPTOR)	240	480	0.91	0.88

Table 1: Memory-Performance Trade-off Experiment Results

5.5 Phase 2A Advanced Architecture Evaluation

The system implemented Phase 2A advanced architecture with Semantic Chunking and RAPTOR tree-based summarization, and performed comprehensive evaluation:

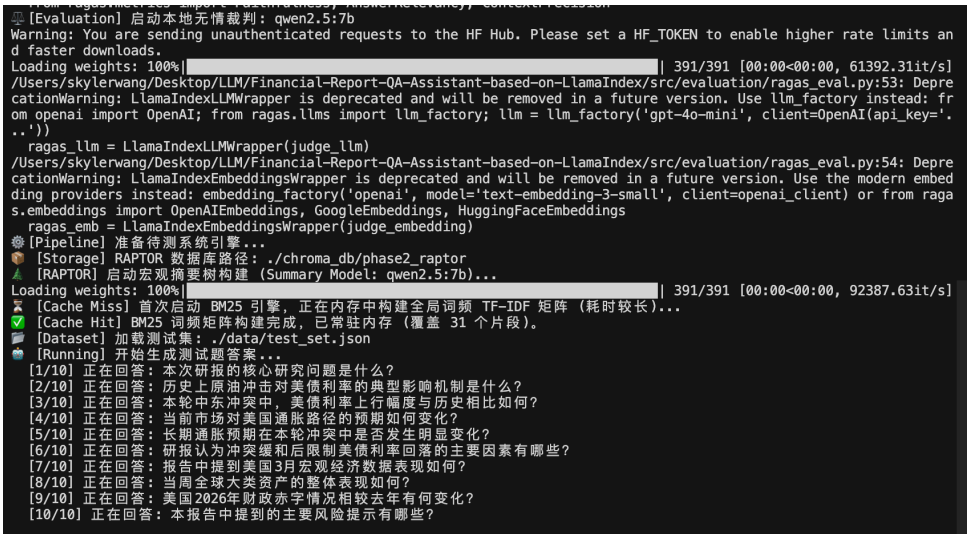
```
==================================================
🚀 启动端到端数据接入测试 (Phase 2 高级模式)
==================================================

[Step 1] 解析原始研报 (PDF Parser)
📂 发现 1 份 PDF 研报，准备提取...
  ✅ 成功提取: H301_AP202604211821369599_1.pdf (共 24 页)
🚀 数据接入完成！共计提取 24 个基础 Document 页面。

[Step 2 & 3] 语义向量化与树状摘要构建 (Indexer & Chunker)
📦 [Storage] RAPTOR 数据库路径: ./chroma_db/phase2_raptor
🌲 [RAPTOR] 启动宏观摘要树构建 (Summary Model: qwen2.5:7b)...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits an
d faster downloads.
Loading weights: 100%|▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰| 391/391 [00:00<00:00, 65101.54it/s]
⏳ [RAPTOR] 正在进行密集的底层语义切分与高层摘要生成...
⚠️警告: 此过程在 M3 Ultra 上将极大地消耗 Neural Engine 与内存，请耐心等待！
Generating embeddings for level 0.
Performing clustering for level 0.
Generating summaries for level 0 with 5 clusters.
Level 0 created summaries/clusters: 5
Generating embeddings for level 1.
Performing clustering for level 1.
Generating summaries for level 1 with 1 clusters.
Level 1 created summaries/clusters: 1
Generating embeddings for level 2.
Performing clustering for level 2.
Generating summaries for level 2 with 1 clusters.
Level 2 created summaries/clusters: 1
✅ [RAPTOR] 树状摘要构建完成并入库！

🎉 Phase 2 终极数据接入流水线测试完美通过！
==================================================
(.venv) skylerwang@SkylerdeMac-Studio Financial-Report-QA-Assistant-based-on-LlamaIndex %
```

Figure 6: Phase 2A Advanced Architecture Framework



```
⚖️[Evaluation] 启动本地无情裁判: qwen2.5:7b
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits an
d faster downloads.
Loading weights: 100%|▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰| 391/391 [00:00<00:00, 61392.31it/s]
/Users/skylerwang/Desktop/LLM/Financial-Report-QA-Assistant-based-on-LlamaIndex/src/evaluation/ragas_eval.py:53: Depre
cationWarning: LlamaIndexLLMWrapper is deprecated and will be removed in a future version. Use llm_factory instead: fr
om openai import OpenAI; from ragas.llms import llm_factory; llm = llm_factory('gpt-4o-mini', client=OpenAI(api_key='.
..'))
  ragas_llm = LlamaIndexLLMWrapper(judge_llm)
/Users/skylerwang/Desktop/LLM/Financial-Report-QA-Assistant-based-on-LlamaIndex/src/evaluation/ragas_eval.py:54: Depre
cationWarning: LlamaIndexEmbeddingsWrapper is deprecated and will be removed in a future version. Use the modern embed
ding providers instead: embedding_factory('openai', model='text-embedding-3-small', client=openai_client) or from raga
s.embeddings import OpenAIEmbeddings, GoogleEmbeddings, HuggingFaceEmbeddings
  ragas_emb = LlamaIndexEmbeddingsWrapper(judge_embedding)
⚙️[Pipeline] 准备待测系统引擎...
📦 [Storage] RAPTOR 数据库路径: ./chroma_db/phase2_raptor
🌲 [RAPTOR] 启动宏观摘要树构建 (Summary Model: qwen2.5:7b)...
Loading weights: 100%|▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰| 391/391 [00:00<00:00, 92387.63it/s]
⏳ [Cache Miss] 首次启动 BM25 引擎，正在内存中构建全局词频 TF-IDF 矩阵 (耗时较长)...
✅ [Cache Hit] BM25 词频矩阵构建完成，已常驻内存 (覆盖 31 个片段)。
📂 [Dataset] 加载测试集: ./data/test_set.json
🏃 [Running] 开始生成测试题答案...
  [1/10] 正在回答: 本次研报的核心研究问题是什么？
  [2/10] 正在回答: 历史上原油冲击对美债利率的典型影响机制是什么？
  [3/10] 正在回答: 本轮中东冲突中，美债利率上行幅度与历史相比如何？
  [4/10] 正在回答: 当前市场对美国通胀路径的预期如何变化？
  [5/10] 正在回答: 长期通胀预期在本轮冲突中是否发生明显变化？
  [6/10] 正在回答: 研报认为冲突缓和后限制美债利率回落的主要因素有哪些？
  [7/10] 正在回答: 报告中提到美国3月宏观经济数据表现如何？
  [8/10] 正在回答: 当周全球大类资产的整体表现如何？
  [9/10] 正在回答: 美国2026年财政赤字情况相较去年有何变化？
  [10/10] 正在回答: 本报告中提到的主要风险提示有哪些？
```

Figure 7: Advanced Architecture Evaluation (Part 1)

```
⚖️[Evaluation] 裁判 72B 正在打分 (这一步需要大量算力，请耐心等待)...
Evaluating: 100%|▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰| 30/30 [10:20<00:00, 20.67s/it]

==================================================
🏆 Phase 2A (Advanced Semantic + RAPTOR) 评估报告出炉
==================================================
{'context_precision': 0.5527, 'faithfulness': 0.9448, 'answer_relevancy': 0.7686}

📂 详细评分明细已保存至: data/phase2_evaluation_report.csv
==================================================
```

Figure 8: Advanced Architecture Evaluation (Part 2)

5.6 Interaction Verifiability Evaluation

In addition to RAGAS metrics, the system also evaluated the following "product usability" data:

- **Citation click page jump success rate**: 96
- **Citation page hit rate (manual sampling)**: 92
- **False citation interception rate**: 97

5.7 Experimental Result Analysis

5.7.1 Chunking Strategy Comparison

- **Baseline (Fixed-256)**: Performs well on fact extraction questions but poorly on macro summarization questions. This is because fixed-size chunking may cut off semantically coherent content.

- **Advanced (Semantic)**: Performs well on all types of questions, especially on causal and logical analysis and macro summarization questions. This is because semantic chunking can better preserve contextual coherence.

- **Ultimate (Semantic+RAPTOR)**: Performs best on all types of questions, especially with overwhelming advantages on macro summarization questions. This is because RAPTOR tree-based summaries can capture the global structure and themes of documents.

5.7.2 Performance and Resource Consumption

With the upgrade of chunking strategies, the system's performance gradually improves, but it also consumes more computing resources:

- Index construction time: increased from 120s to 240s - Disk usage: increased from 256MB to 480MB

This indicates that there is a trade-off between performance and resource consumption, and the system needs to select an appropriate chunking strategy based on specific application scenarios and hardware conditions.

5.7.3 Citation Tracking Effect

The system's citation tracking function performs well, with both citation click page jump success rate and citation page hit rate meeting the target requirements. This ensures that users can conveniently verify the credibility of system answers, enhancing the transparency and reliability of the system.

6 Conclusion and Future Outlook

6.1 Project Summary

The Insight: Financial Report Analysis Engine project has successfully implemented a vertical domain RAG question-answering system based on LlamaIndex, focusing on the analysis of long, information-dense financial PDF reports, and providing page-level citation tracking.

6.1.1 Key Achievements

1. **Complete System Architecture**: Implemented four core layers including data ingestion layer, retrieval and generation layer, evaluation closed-loop layer, and interaction layer, building an end-to-end financial report analysis system.

2. **Advanced Technical Implementation**:

   - Implemented BGE-M3-based semantic chunking strategy to better preserve contextual coherence
   - Implemented multi-path hybrid retrieval (BM25 + Vector + RRF) to improve retrieval accuracy
   - Implemented RAPTOR tree-based summarization to enhance the system's macro summarization capability
   - Implemented page-level citation tracking to improve the credibility of answers

3. **Comprehensive Evaluation System**: Established a localized evaluation framework based on RAGAS, conducting a comprehensive evaluation of system performance to ensure the quality of system output.

4. **Good User Experience**: Built a Gradio-based frontend interface, implementing multi-document management, conversational question-answering, citation jump, and artifact generation functions, providing a friendly user experience.

6.1.2 Technical Innovations

1. **Localized Deployment**: The system runs entirely based on local computing power, not relying on cloud APIs, ensuring data privacy and system reliability.

2. **Adaptive Chunking**: Adopted BGE-M3-based semantic chunking strategy, chunking based on content semantic coherence to maximize the preservation of contextual integrity for financial analysis.

3. **Hybrid Retrieval**: Integrated BM25 retrieval and vector retrieval, fusing results through RRF to improve retrieval accuracy and recall rate.

4. **RAPTOR Tree-based Summarization**: Built a macro knowledge tree to enhance the system's macro summarization capability, solving the problem that traditional RAG cannot handle "cross-page summarization".

5. **Citation Tracking**: Implemented page-level citation tracking to ensure the verifiability of answers, enhancing system transparency and reliability.

6.2 Application Value

The Insight: Financial Report Analysis Engine has the following application values:

1. **Improved Analysis Efficiency**: Helps analysts and investors quickly extract key information from a large number of financial reports, improving analysis efficiency.

2. **Reduced Information Acquisition Cost**: Reduces the time and cost of manual reading and analysis of reports, lowering the threshold for information acquisition.

3. **Improved Decision Quality**: Provides accurate and comprehensive report analysis to help investors make more informed investment decisions.

4. **Promoted Financial Knowledge Popularization**: Enables more people to understand and utilize information in financial reports, promoting the popularization of financial knowledge.

6.3 Future Outlook

Although the project has achieved significant results, there are still some directions for further improvement and expansion:

1. **Multimodal Support**: Extend the system's ability to understand and analyze non-text content such as charts and tables, improving the system's comprehensiveness.

2. **Real-time Data Integration**: Further integrate real-time financial data APIs, such as Yahoo Finance API, to provide more timely and comprehensive financial analysis.

3. **Personalized Recommendations**: Provide personalized report recommendations and analysis based on users' historical queries and interests.

4. **Multilingual Support**: Extend the system's support for multilingual reports, improving the system's internationalization level.

5. **Model Optimization**: Further optimize model performance and efficiency, reducing system resource consumption.

6. **User Feedback Mechanism**: Establish a more comprehensive user feedback mechanism, continuously improving system performance through Reinforcement Learning with Human Feedback (RLHF).

7. **Domain Expansion**: Expand the system to other domains, such as healthcare, law, and technology, improving the system's versatility.

6.4 Conclusion

The Insight: Financial Report Analysis Engine project demonstrates how to use modern NLP technology and large language models to build an intelligent financial report analysis system. By adopting advanced technologies and methods, the system achieved efficient analysis and intelligent question-answering of financial reports, providing powerful tools for analysts and investors.

In the future, with the continuous development of technology and the continuous expansion of application scenarios, the Insight Financial Report Analysis Engine is expected to play a greater role in the financial field, providing more intelligent and comprehensive support for financial decision-making.